DATA 260 - Homework 8: Kafka Request/Response, a Three-Agent Pipeline and Failure Behavior

GitHub repository: https://github.com/pragnayap/data260-3170

Section 0 - Configuration

Value	Definition	Mine
SID4	last 4 of SJSU student ID	3170
PORT_BASE	8000 + (SID4 mod 900)	8470
PREFIX	"s" + SID4	s3170
SEED	SID4	3170
VERIFY_SEED	260000 + SID4	263170
DOMAIN_ID	SID4 mod 8	2 - Municipal Transit Incidents (entity: Transit Incident)
Hardware		Intel Core i9-9880H (x86_64), 16 GB RAM, macOS 15, Docker Desktop
Local model		qwen3:8b via Ollama, CPU-only (Part 2 agents); Kafka: apache/kafka:3.8.0 (KRaft) + confluent-kafka 2.16.1
Tagged commit hash	tag hw8	4815669

Overview - what have been built

Part	What have been built	Result
1	Entity worker + FastAPI wrapper that talk only through two Kafka topics (CREATE_INCIDENT / GET_INCIDENT); homework_demo.py; Docker Compose with Kafka, worker and API.	create → get round-trips; correlation_id preserved in every reply, including errors
2	Planner, Writer and Reviewer as separate processes over four topics; send_question.py.	one question reached s3170_final, status approved, correlation_id unchanged
3	Duplicate replay, consumer killed mid-work, poison message, ordering - each run before and after its fix.	4 of 4 scenarios reproduce the failure and show the fix working (2 consecutive full passes)

Part 1 - Backend Service Request/Response over Kafka

What was done

- Primary entity: Transit Incident (routeId, location, submitterEmail, description > 25 characters, category, agreeTerms). So the operations are CREATE_INCIDENT and GET_INCIDENT, and the id field is incident_id.
- Exactly two topics: s3170_entity_requests (client/API → worker) and s3170_entity_responses (worker → whoever sent the request).
- The client creates the correlation_id once; the worker copies it unchanged into every response, including errors (unknown operation, unknown id, validation failures, even a malformed request).

- The worker keeps entities in a plain dict, generates incident_id with uuid4, validates every field, and commits its Kafka offset only after the reply is delivered.
- FastAPI wrapper (port 8470): POST /incidents and GET /incidents/{id} become Kafka requests; the HTTP reply is the worker's response for the same correlation_id (also returned in an X-Correlation-Id header).
- Docker Compose runs Kafka (single-node KRaft), the worker and the API; homework_demo.py runs the 10-step flow and prints the worker container's own log lines for each request.

Topics and message schemas

Topic	Direction	Partitions
s3170_entity_requests	client / API → worker	1
s3170_entity_responses	worker → client / API (named by reply_to)	1

Code

```
def validate(data: Any) -> str | None:
    """Return a clean error message for missing / empty / invalid fields, else None."""
    if not isinstance(data, dict):
        return "data must be an object"
    for field in REQUIRED:
        if field not in data:
            return f"Missing required field: {field}"
    for field in ("routeId", "location", "submitterEmail", "description", "category"):
        if not isinstance(data[field], str) or not data[field].strip():
            return f"Field {field} must be a non-empty string"
    if not EMAIL.match(data["submitterEmail"].strip()):
        return "Field submitterEmail must be a valid email address"
    if len(data["description"].strip()) <= 25:
        return "Field description must be longer than 25 characters"
    if data["category"] not in CATEGORIES:
        return f"Field category must be one of: {', '.join(CATEGORIES)}"
    if data["agreeTerms"] is not True:
        return "Field agreeTerms must be true"
    return None


def handle(request: dict[str, Any]) -> dict[str, Any]:
    """Process one request dict and return the response body (without correlation_id)."""
    operation = request.get("operation")
    data = request.get("data")

    if operation == "CREATE_INCIDENT":
        error = validate(data)
        if error:
            return {"success": False, "error": error}
        incident_id = new_id()
        ENTITIES[incident_id] = {"incident_id": incident_id,
                                 **{k: (data[k].strip() if isinstance(data[k], str) else data[k])
                                    for k in REQUIRED}}
        return {"success": True, "incident_id": incident_id, "message": "Incident created"}

    if operation == "GET_INCIDENT":
        incident_id = data.get("incident_id") if isinstance(data, dict) else None
        if not isinstance(incident_id, str) or not incident_id.strip():
            return {"success": False, "error": "Missing required field: incident_id"}
        entity = ENTITIES.get(incident_id)
        if entity is None:
            return {"success": False, "error": "Incident not found"}
        return {"success": True, "incident": entity}

    return {"success": False, "error": "Unknown operation"}
```

```
if not correlation_id or not isinstance(request.get("data"), dict | type(None)):
    body = {"success": False, "error": "Request must include correlation_id and operation"}
else:
    log("WORKER", f"PROCESS correlation_id={correlation_id} operation={operation} data={request.get('data')}")
    body = handle(request)
```

```
target = reply_to if isinstance(reply_to, str) and reply_to.startswith(f"{PREFIX}_") else RESPONSES
response = {"correlation_id": correlation_id, **body}
partition, offset = produce(producer, target, response, key=correlation_id)
log("WORKER", f"REPLY topic={target} partition={partition} offset={offset} "
              f"correlation_id={correlation_id} success={body['success']} "
              f"{'error=' + body['error'] if not body['success'] else ''}".rstrip())
consumer.commit(message=msg, asynchronous=False)   # only after the reply is on the broker
```

```
def call_worker(operation: str, data: dict[str, Any]) -> tuple[str, dict[str, Any]]:
    """Send one request through Kafka and block until its response arrives (or time out)."""
    correlation_id = new_id()                  # created once, here, and never changed
    PENDING[correlation_id] = {"event": threading.Event(), "response": None}
    request = {"correlation_id": correlation_id, "reply_to": RESPONSES, "operation": operation, "data": data}
    start = time.time()
    try:
        partition, offset = produce(_producer, REQUESTS, request, key=correlation_id)
        log("API", f"SEND  topic={REQUESTS} partition={partition} offset={offset} "
                   f"correlation_id={correlation_id} operation={operation}")
        slot = PENDING[correlation_id]
        if not slot["event"].wait(TIMEOUT_S):
            log("API", f"TIMEOUT correlation_id={correlation_id} after {TIMEOUT_S}s")
            raise HTTPException(504, {"correlation_id": correlation_id, "error": "Worker did not answer in time"})
        response = slot["response"]
    finally:
        PENDING.pop(correlation_id, None)
    log("API", f"CLIENT-RESPONSE correlation_id={correlation_id} success={response.get('success')} "
               f"in {int((time.time() - start) * 1000)} ms")
    return correlation_id, response
```

```
def roundtrip(producer, consumer, operation: str, data: dict[str, Any]) -> tuple[dict, dict]:
    """Send one request and wait for the response carrying the same correlation_id."""
    correlation_id = new_id()
    request = {"correlation_id": correlation_id, "reply_to": RESPONSES, "operation": operation, "data": data}
    partition, offset = produce(producer, REQUESTS, request, key=correlation_id)
    log("CLIENT", f"SEND  topic={REQUESTS} partition={partition} offset={offset} "
                  f"correlation_id={correlation_id} operation={operation}")
    log("CLIENT", f"      message: {json.dumps(request)}")
    deadline = time.time() + 30
    while time.time() < deadline:
        msg = consumer.poll(0.5)
        if msg is None or msg.error():
            continue
        response = decode(msg.value())
        if response.get("correlation_id") == correlation_id:
            log("CLIENT", f"RECV  topic={msg.topic()} partition={msg.partition()} offset={msg.offset()} "
                          f"correlation_id={response['correlation_id']} success={response.get('success')}")
            log("CLIENT", f"      message: {json.dumps(response)}")
            return request, response
    raise TimeoutError(f"no response for correlation_id={correlation_id} - is the worker running?")
```

Evidence

```
Pragnaya Priyadarshini HW8
NAMES          IMAGE                  STATUS                   PORTS
s3170_api      data260-3170-api       Up 2 hours               0.0.0.0:8470->8470/tcp, [::]:8470->8470/tcp
s3170_worker   data260-3170-worker    Up 2 hours
s3170_kafka    apache/kafka:3.8.0     Up 2 hours (healthy)     0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
s3170_mongo    mongo:7.0              Up 3 hours               0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
08:44:31.709 [WORKER  ] Consumer is ready and listening...
08:48:22.375 [API     ] FastAPI server started at http://127.0.0.1:8470
(base)
data260-3170 on  hw08-work [!?] via  v3.13.5 on  (us-east-1)  Pragnaya Priyadarshini HW8
❯
```

[Screenshot 1.1: docker ps showing the Kafka, worker and FastAPI containers]

```
Pragnaya Priyadarshini HW8 - homework_demo.py

================================================================================
STEPS 1–7: CREATE_INCIDENT
================================================================================
09:16:05.025 [DEMO    ] STEP 1  one correlation_id is created per request, before sending
09:16:05.026 [DEMO    ] STEP 2–3 send CREATE_INCIDENT to the request topic and log the message
09:16:05.034 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=110 correlation_id=cb446d43-cb6f-4049-b19b-f7d84032dbcf operation=CREATE_INCIDENT
09:16:05.034 [CLIENT  ]       message: {"correlation_id": "cb446d43-cb6f-4049-b19b-f7d84032dbcf", "reply_to": "s3170_entity_responses", "operation": "CREATE_INCIDENT", "data": {"routeId": "Line 22", "location": "Downtown Transit Center",
"submitterEmail": "rider@example.com", "description": "Bus arrived 40 minutes late with no announcement or updated signage at the stop.", "category": "Delay", "agreeTerms": true}}
09:16:05.037 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=110 correlation_id=cb446d43-cb6f-4049-b19b-f7d84032dbcf success=True
09:16:05.037 [CLIENT  ]       message: {"correlation_id": "cb446d43-cb6f-4049-b19b-f7d84032dbcf", "success": true, "incident_id": "63440bf9-8e03-4318-a77b-049b0c14a5eb", "message": "Incident created"}
09:16:05.037 [DEMO    ] STEP 4–5 worker received, processed and replied (its own log lines):
      (worker container) 09:16:05.033 [WORKER  ] RECV  topic=s3170_entity_requests partition=0 offset=110 correlation_id=cb446d43-cb6f-4049-b19b-f7d84032dbcf operation=CREATE_INCIDENT
      (worker container) 09:16:05.033 [WORKER  ] PROCESS correlation_id=cb446d43-cb6f-4049-b19b-f7d84032dbcf operation=CREATE_INCIDENT data={'routeId': 'Line 22', 'location': 'Downtown Transit Center', 'submitterEmail': 'rider@example.com
', 'description': 'Bus arrived 40 minutes late with no announcement or updated signage at the stop.', 'category': 'Delay', 'agreeTerms': True}
      (worker container) 09:16:05.034 [WORKER  ] REPLY topic=s3170_entity_responses partition=0 offset=110 correlation_id=cb446d43-cb6f-4049-b19b-f7d84032dbcf success=True
09:16:05.580 [DEMO    ] STEP 6  response received by the client (logged above)
09:16:05.580 [DEMO    ] STEP 7  extracted incident_id=63440bf9-8e03-4318-a77b-049b0c14a5eb
```

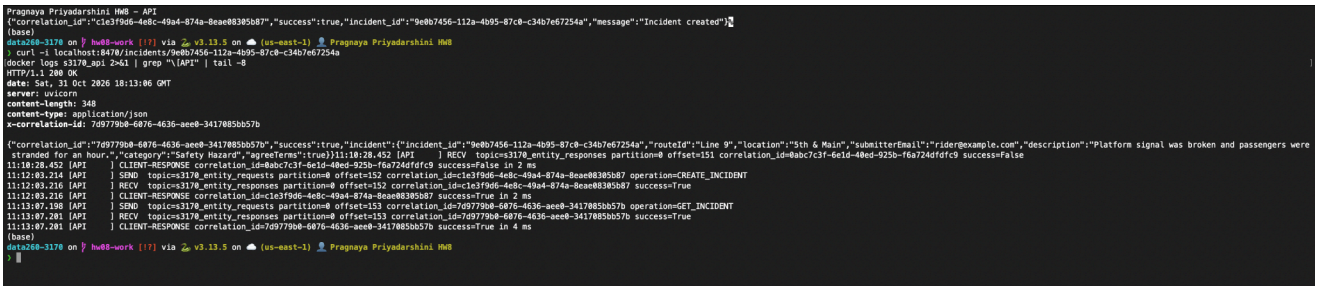
[Screenshot 1.2: homework_demo.py - CREATE_INCIDENT request, response, extracted incident_id, success status and correlation_id]

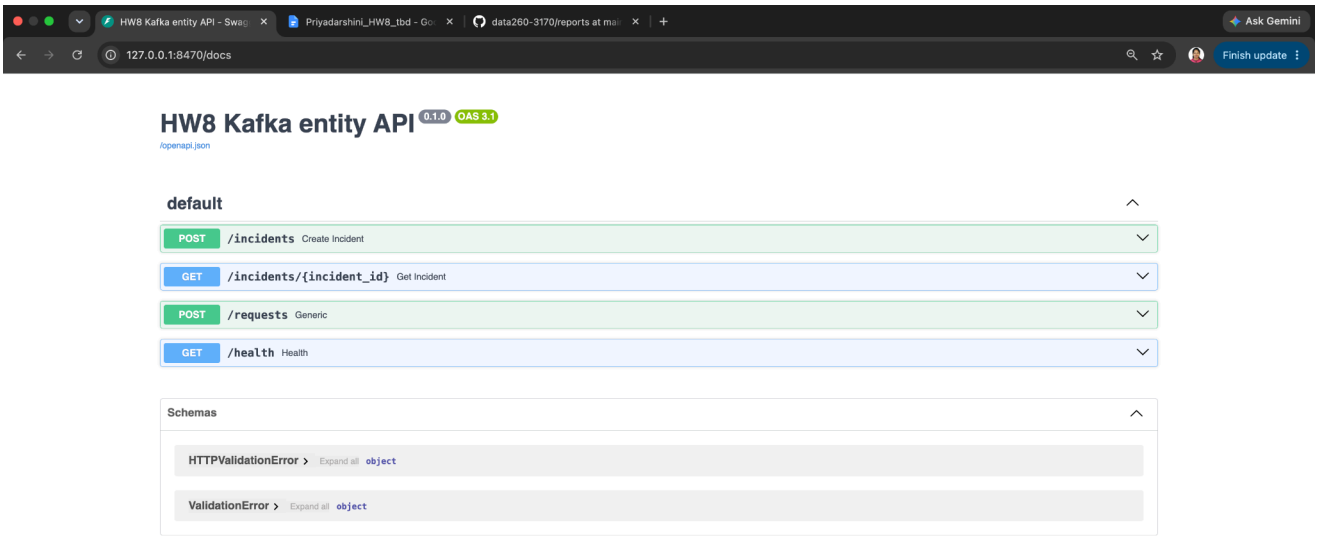
```
STEPS 8–10: GET_INCIDENT with that incident_id
================================================================================
09:16:05.580 [DEMO    ] STEP 8  send GET_INCIDENT
09:16:05.582 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=111 correlation_id=8bb0f483-5bfd-42be-84ba-25aa2f3651b7 operation=GET_INCIDENT
09:16:05.582 [CLIENT  ]       message: {"correlation_id": "8bb0f483-5bfd-42be-84ba-25aa2f3651b7", "reply_to": "s3170_entity_responses", "operation": "GET_INCIDENT", "data": {"incident_id": "63440bf9-8e03-4318-a77b-049b0c14a5eb"}}
09:16:05.584 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=111 correlation_id=8bb0f483-5bfd-42be-84ba-25aa2f3651b7 success=True
09:16:05.584 [CLIENT  ]       message: {"correlation_id": "8bb0f483-5bfd-42be-84ba-25aa2f3651b7", "success": true, "incident": {"incident_id": "63440bf9-8e03-4318-a77b-049b0c14a5eb", "routeId": "Line 22", "location": "Downtown Transit Cen
ter", "submitterEmail": "rider@example.com", "description": "Bus arrived 40 minutes late with no announcement or updated signage at the stop.", "category": "Delay", "agreeTerms": true}}
09:16:05.584 [DEMO    ] STEP 9  worker received, processed and replied (its own log lines):
      (worker container) 09:16:05.581 [WORKER  ] RECV  topic=s3170_entity_requests partition=0 offset=111 correlation_id=8bb0f483-5bfd-42be-84ba-25aa2f3651b7 operation=GET_INCIDENT
      (worker container) 09:16:05.581 [WORKER  ] PROCESS correlation_id=8bb0f483-5bfd-42be-84ba-25aa2f3651b7 operation=GET_INCIDENT data={'incident_id': '63440bf9-8e03-4318-a77b-049b0c14a5eb'}
      (worker container) 09:16:05.582 [WORKER  ] REPLY topic=s3170_entity_responses partition=0 offset=111 correlation_id=8bb0f483-5bfd-42be-84ba-25aa2f3651b7 success=True
09:16:06.132 [DEMO    ] STEP 10 retrieved entity:
{
  "incident_id": "63440bf9-8e03-4318-a77b-049b0c14a5eb",
  "routeId": "Line 22",
  "location": "Downtown Transit Center",
  "submitterEmail": "rider@example.com",
  "description": "Bus arrived 40 minutes late with no announcement or updated signage at the stop.",
  "category": "Delay",
  "agreeTerms": true
}
09:16:06.132 [DEMO    ] correlation_id sent     = 8bb0f483-5bfd-42be-84ba-25aa2f3651b7
09:16:06.132 [DEMO    ] correlation_id received = 8bb0f483-5bfd-42be-84ba-25aa2f3651b7
09:16:06.132 [DEMO    ] correlation_id preserved: True
```

[Screenshot 1.3: Kafka message-flow logs for GET_INCIDENT - sent to s3170_entity_requests, received and processed by the worker, reply on s3170_entity_responses, response received]

```
Pragnaya Priyadarshini HW8 - homework_demo.py
STEPS 1–7: CREATE_INCIDENT
09:19:25.234 [DEMO    ] STEP 1  one correlation_id is created per request, before sending
09:19:25.234 [DEMO    ] STEP 2–3 send CREATE_INCIDENT to the request topic and log the message
09:19:25.241 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=115 correlation_id=ecc805f7-12d7-4d10-b977-8b8ba32b2418 operation=CREATE_INCIDENT
09:19:25.245 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=115 correlation_id=ecc805f7-12d7-4d10-b977-8b8ba32b2418 success=True
09:19:25.245 [DEMO    ] STEP 4–5 worker received, processed and replied (its own log lines):
      (worker container) 09:19:25.240 [WORKER  ] RECV  topic=s3170_entity_requests partition=0 offset=115 correlation_id=ecc805f7-12d7-4d10-b977-8b8ba32b2418 operation=CREATE_INCIDENT
      (worker container) 09:19:25.240 [WORKER  ] PROCESS correlation_id=ecc805f7-12d7-4d10-b977-8b8ba32b2418 operation=CREATE_INCIDENT data={'routeId': 'Line 22', 'location': 'Downtown Transit Center', 'submitterEmail': 'rider@example.com
', 'description': 'Bus arrived 40 minutes late with no announcement or updated signage at the stop.', 'category': 'Delay', 'agreeTerms': True}
      (worker container) 09:19:25.242 [WORKER  ] REPLY topic=s3170_entity_responses partition=0 offset=115 correlation_id=ecc805f7-12d7-4d10-b977-8b8ba32b2418 success=True
09:19:25.793 [DEMO    ] STEP 6  response received by the client (logged above)
09:19:25.794 [DEMO    ] STEP 7  extracted incident_id=6461a242-9798-47bd-adf9-90f8a82adda0
STEPS 8–10: GET_INCIDENT with that incident_id
09:19:25.794 [DEMO    ] STEP 8  send GET_INCIDENT
09:19:25.796 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=116 correlation_id=ddd62543-8740-4d08-b299-779c5da789b9 operation=GET_INCIDENT
09:19:25.798 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=116 correlation_id=ddd62543-8740-4d08-b299-779c5da789b9 success=True
09:19:25.799 [DEMO    ] STEP 9  worker received, processed and replied (its own log lines):
      (worker container) 09:19:25.795 [WORKER  ] RECV  topic=s3170_entity_requests partition=0 offset=116 correlation_id=ddd62543-8740-4d08-b299-779c5da789b9 operation=GET_INCIDENT
      (worker container) 09:19:25.795 [WORKER  ] PROCESS correlation_id=ddd62543-8740-4d08-b299-779c5da789b9 operation=GET_INCIDENT data={'incident_id': '6461a242-9798-47bd-adf9-90f8a82adda0'}
      (worker container) 09:19:25.796 [WORKER  ] REPLY topic=s3170_entity_responses partition=0 offset=116 correlation_id=ddd62543-8740-4d08-b299-779c5da789b9 success=True
09:19:26.346 [DEMO    ] STEP 10 retrieved entity:
09:19:26.349 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=117 correlation_id=40689b2c-b79b-4239-9f66-df5f944aa386 operation=GET_INCIDENT
09:19:26.351 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=117 correlation_id=40689b2c-b79b-4239-9f66-df5f944aa386 success=False
09:19:26.353 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=118 correlation_id=24e36431-e273-4a68-ace0-bfff99656773 operation=DELETE_INCIDENT
09:19:26.356 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=118 correlation_id=24e36431-e273-4a68-ace0-bfff99656773 success=False
09:19:26.358 [CLIENT  ] SEND  topic=s3170_entity_requests partition=0 offset=119 correlation_id=b891c6a0-e23b-42de-9ecb-c7c6d53526f4 operation=CREATE_INCIDENT
09:19:26.360 [CLIENT  ] RECV  topic=s3170_entity_responses partition=0 offset=119 correlation_id=b891c6a0-e23b-42de-9ecb-c7c6d53526f4 success=False
(base)
data260-3170/code/hw08 on  hw08-work [!?] via  v3.13.5 on  (us-east-1)  Pragnaya Priyadarshini HW8
❯
```

[Screenshot 1.4: full sequence CREATE_INCIDENT → GET_INCIDENT (send, receive, process, reply, final client response, in order)]





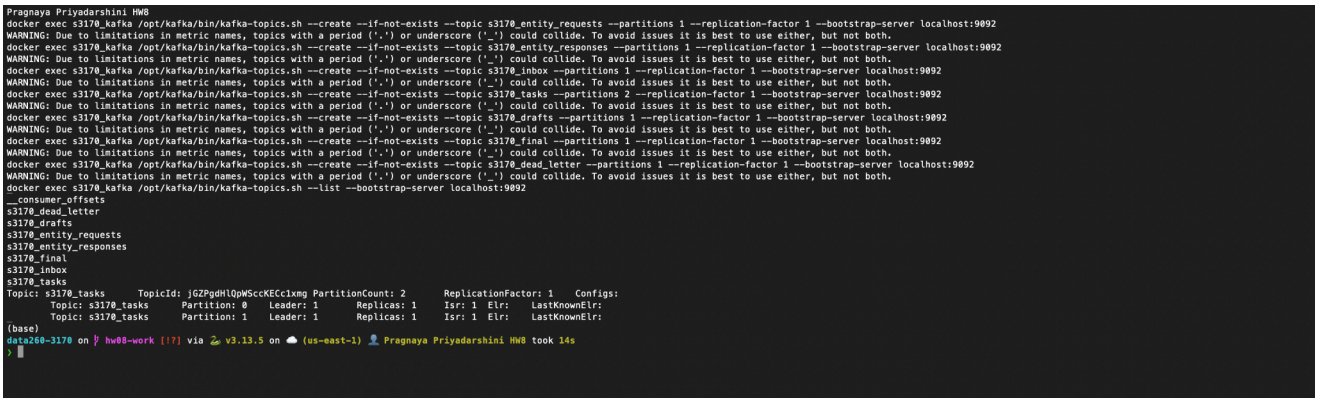
[Screenshot 1.5: the FastAPI wrapper - curl POST /incidents (201) and GET /incidents/{id} (200) with the X-Correlation-Id header, and the API container's own Kafka send/receive log lines]

Part 2 - Three-Agent Kafka Pipeline

What was done

- Planner (reads s3170_inbox → writes s3170_tasks), Writer (tasks → s3170_drafts) and Reviewer (drafts → s3170_final) run as three separate processes. No agent calls another; every hand-off is a Kafka message, keyed by correlation_id.
- The Planner and Writer call the local qwen3:8b (capped at a few hundred tokens, with a client timeout); the Reviewer is rule-based (length and placeholder checks), so the final message is reproducible.
- send_question.py puts a question on s3170_inbox and waits on s3170_final for the message with the same correlation_id.
- The four topics were created with the Kafka CLI (s3170_tasks has 2 partitions, which Part 3 uses for ordering).

Topics



[Screenshot 2.1: the four pipeline topics (plus the others) created/listed with the Kafka CLI, s3170_tasks showing PartitionCount 2]

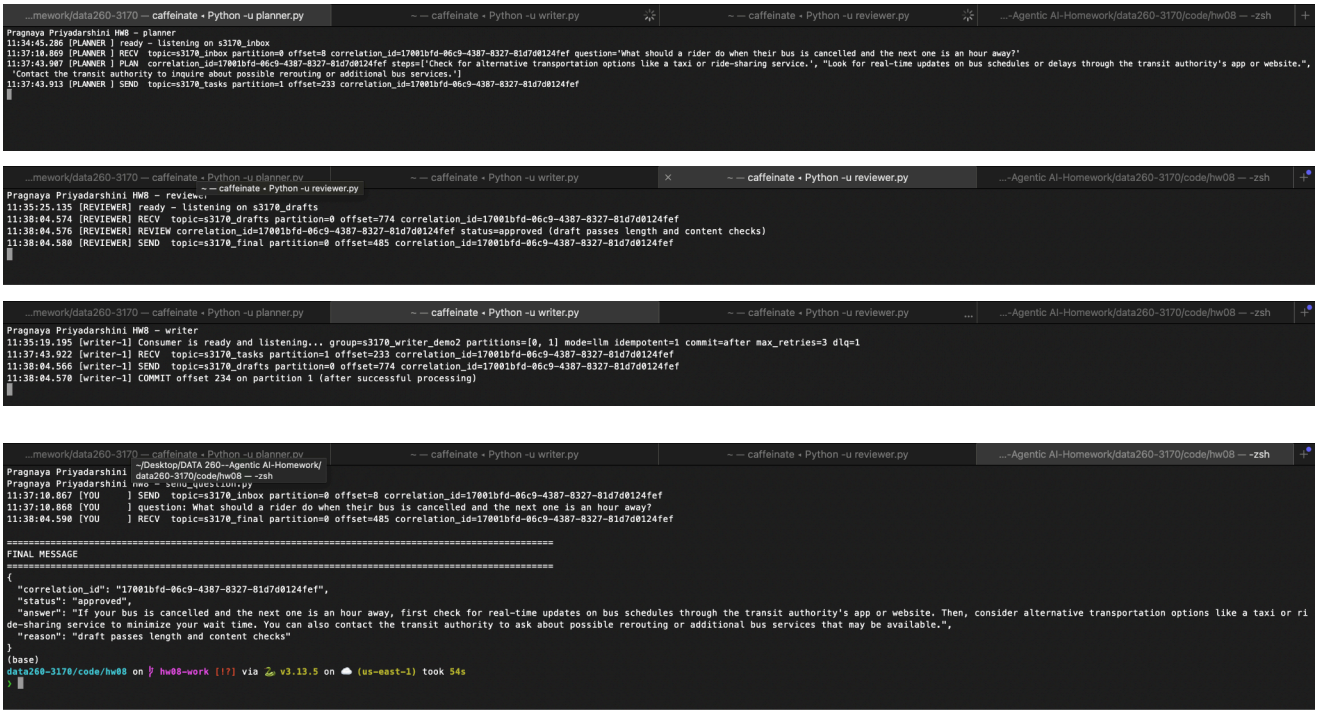
Code

```python
def make_plan(question: str) -> list[str]:
    if os.environ.get("PLANNER_MODE") == "stub":      # fast canned plan for the self-check
        return ["Restate the question", "Give the key facts", "Summarise in two sentences"]
    reply = complete("Break this question into exactly 3 short steps for writing a good answer. "
                     "Reply with only a JSON array of 3 strings.\nQuestion: " + question, max_tokens=150)
    match = re.search(r"\[.*\]", reply)
    try:
        steps = [str(s) for s in json.loads(match.group(0))] if match else []
    except ValueError:
        steps = []
    return steps[:5] or ["Restate the question", "Give the key facts", "Summarise in two sentences"]
```

```python
def review(draft: str) -> tuple[str, str]:
    text = (draft or "").strip()
    if len(text) < MIN_CHARS:
        return "rejected", f"draft too short ({len(text)} < {MIN_CHARS} characters)"
    if len(text) > MAX_CHARS:
        return "rejected", f"draft too long ({len(text)} > {MAX_CHARS} characters)"
    for phrase in BAD_PHRASES:
        if phrase in text.lower():
            return "rejected", f"draft contains placeholder text: {phrase!r}"
    return "approved", "draft passes length and content checks"
```

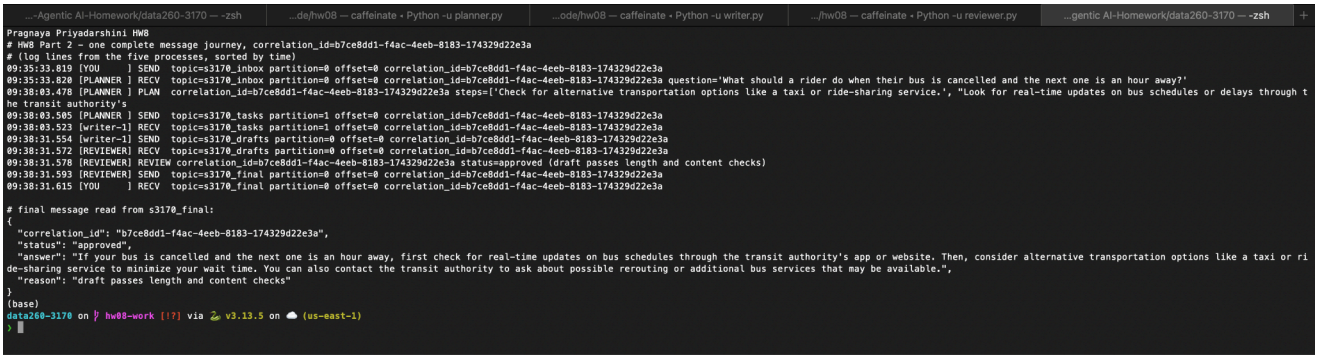
```python
def work(task: dict[str, Any]) -> dict[str, Any]:
    slow = task.get("simulate_slow_ms")
    if slow:
        log(WRITER_ID, f"WORKING correlation_id={task.get('correlation_id')} - busy for {slow} ms "
                       "(kill this process now to simulate a crash mid-work)")
        time.sleep(slow / 1000)
    if task.get("work_ms"):
        time.sleep(task["work_ms"] / 1000)
    text = write_draft(task)
    passthrough = {k: task[k] for k in ("entity", "seq", "run") if k in task}
    return {"correlation_id": task["correlation_id"], "question": task.get("question"), "draft": text,
            "writer": WRITER_ID, **passthrough}
```

```
plan = make_plan(question)
log("PLANNER", f"PLAN  correlation_id={correlation_id} steps={plan}")
partition, offset = produce(producer, TASKS, {"correlation_id": correlation_id, "question": question,
                                              "plan": plan}, key=correlation_id)
log("PLANNER", f"SEND  topic={TASKS} partition={partition} offset={offset} correlation_id={correlation_id}")
consumer.commit(message=msg, asynchronous=False)
```

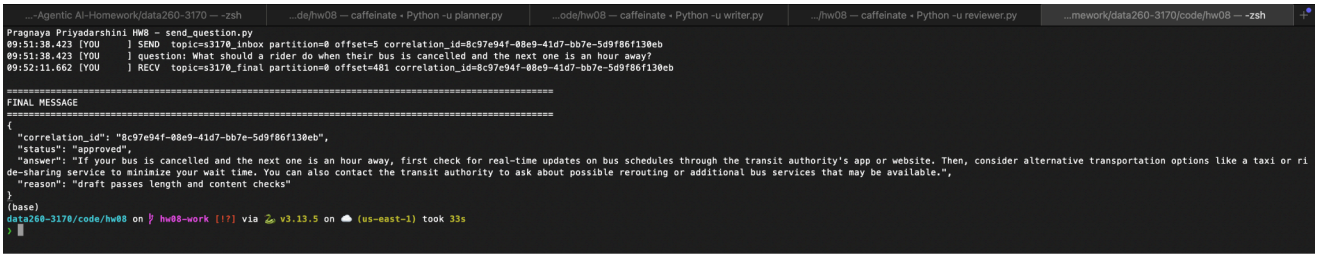


[Screenshot 2.2: the four terminals - planner.py, writer.py, reviewer.py running, and send_question.py sending the question]

One complete message journey



[Screenshot 2.3: the log lines of one message journey from s3170_inbox to s3170_final, one stage per line]



[Screenshot 2.4: the final JSON message (status and answer) read from s3170_final]

How Kafka helped the agents coordinate

Kafka decoupled the three agents in time, in failure and in speed. The Planner finished and moved on while the slower Writer was still generating; the broker held the plan until the Writer was ready, so no agent had to wait on, or even know about, another. Each topic is a durable buffer with its own offsets, so an agent that is restarted resumes where it stopped instead of losing work (Part 3). The correlation_id carried in every message tied the whole journey together across five processes, so one grep reconstructs it. And because the hand-offs are messages, any stage can be scaled out (the tasks topic has two partitions) or replaced without touching the others.

Part 3 - Failure Behavior

What was done

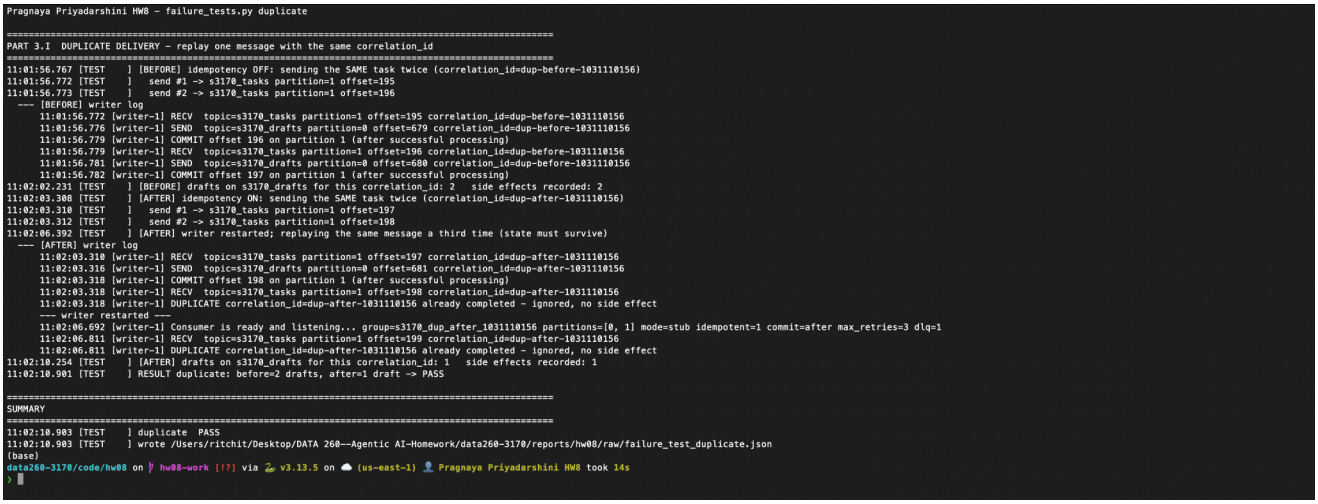
- Every scenario runs against the real broker with real Writer processes (the same writer.py as Part 2, with one safeguard switched off for "before" and on for "after"); failures are injected with SIGKILL, replays and bad payloads.
- The failure tests use stub drafts (instant) because they test Kafka behavior, not model quality. Two consecutive full runs gave identical outcomes.

Failure table

Failure scenario	Before fix	After fix
Duplicate delivery	same correlation_id sent 2× → 2 drafts, 2 side effects	sent 3× (incl. a replay after a restart) → 1 draft, 1 side effect; 2 replays logged DUPLICATE and ignored
Consumer killed mid-work	auto-commit: offset advanced before the work finished; after SIGKILL + restart the message is never redelivered (0 drafts) - lost	commit after success: lag 1 after the kill; restart redelivers it, completes it, commits, lag 0 (1 draft)
Poison message	unbounded retries: 12+ attempts on one offset; the good message behind it is BLOCKED	3 attempts then s3170_dead_letter (2 bad messages); the good message is processed
Two partitions / two consumers	no key: 3 of 5 incidents processed out of order; 4 of 5 split across both partitions	key = incident: 0 of 5 out of order; 5 of 5 stay in one partition

3.I Duplicate delivery

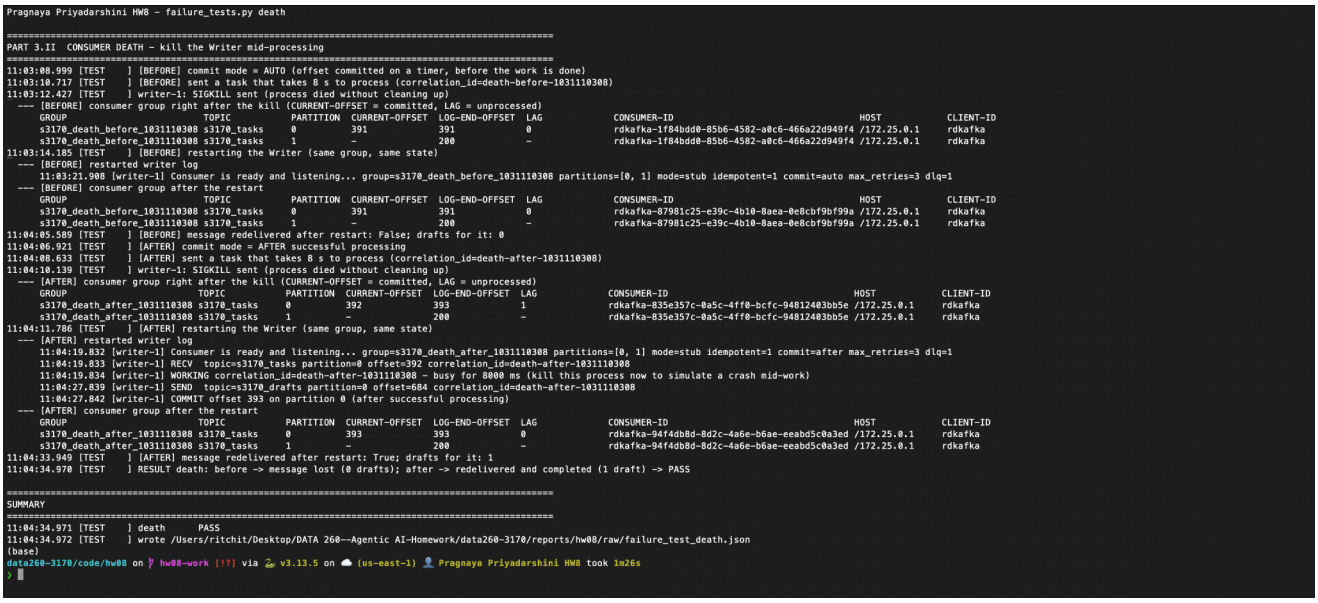
A task is replayed with the same correlation_id. The fix is idempotency: the Writer records each completed correlation_id in an append-only state log (it survives a restart) and ignores a replay.



[Screenshot 3.1: duplicate delivery - before (2 drafts, 2 side effects) and after (1 draft, replay logged as DUPLICATE)]

3.II Consumer death

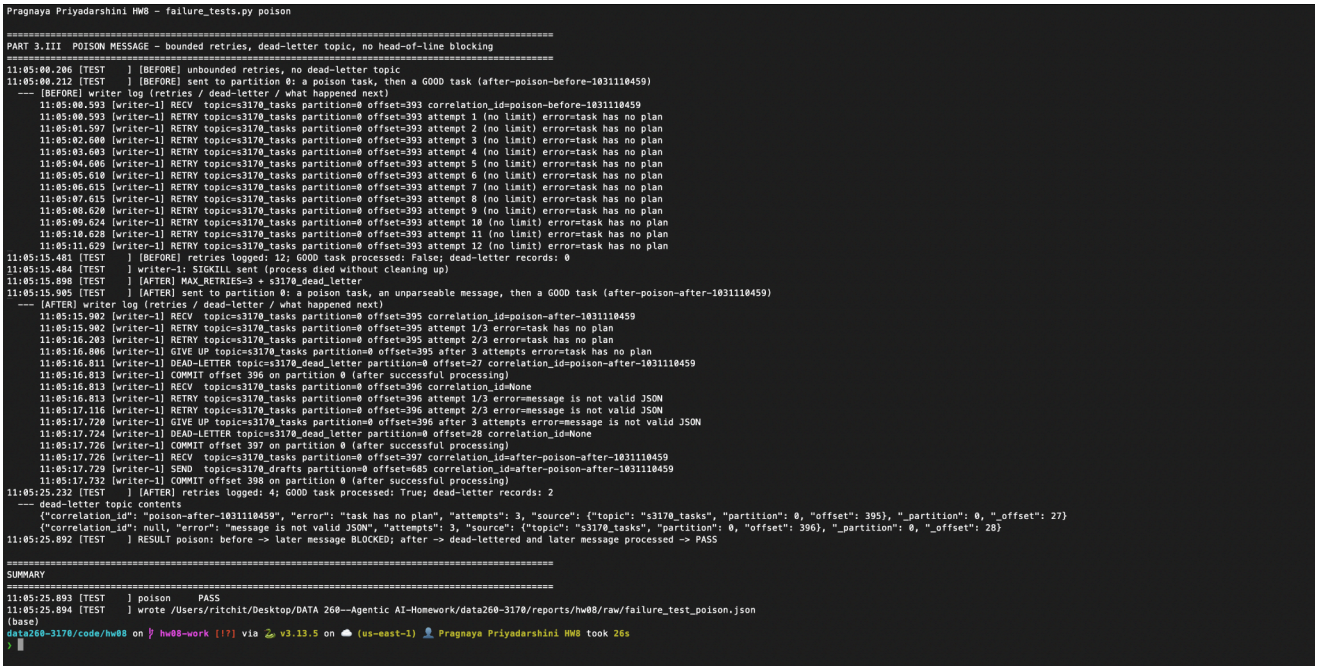
The Writer is killed with SIGKILL while it is processing. Before: auto-commit advances the group's offset on a timer, so the in-flight message is lost. After: the offset is committed only once the draft is delivered, so the broker redelivers it to the restarted Writer.



[Screenshot 3.2: consumer death - the Writer killed mid-work, the consumer-group offsets (committed vs log-end, lag) and the restart; before vs after]

3.III Poison message

A task with no plan always fails. Before: the Writer retries it forever and the partition behind it is blocked. After: at most 3 attempts, then the message is published to s3170_dead_letter with its error and the offset is committed, so later messages flow. An unparseable message is handled the same way.



[Screenshot 3.3: poison message - before (endless retries, later message blocked) and after (3 attempts, s3170_dead_letter, later message processed)]

3.IV Ordering

20 related messages are generated from VERIFY_SEED = 263170 (5 incidents, random per-message work time) and sent through s3170_tasks (2 partitions) with two Writers in one consumer group. Run 1 has no key (messages alternate across the partitions); run 2 uses the incident id as the message key, so Kafka hashes every message for one incident to the same partition.

```
    if keyed:
        partition, _ = produce(producer, TASKS, m, key=m["entity"])
    else:
        partition, _ = produce(producer, TASKS, m, partition=i % 2)
    sent_partition[m["entity"]].add(partition)
```

Run 1 - no key: order in which the 20 messages were processed

#	writer	partition	incident	seq
1	writer-A	1	INC-8446	1
2	writer-B	0	INC-5457	1
3	writer-B	0	INC-2393	1
4	writer-A	1	INC-5457	2
5	writer-A	1	INC-8206	1
6	writer-B	0	INC-4412	1
7	writer-A	1	INC-2393	3
8	writer-A	1	INC-5457	3
9	writer-B	0	INC-2393	2
10	writer-A	1	INC-5457	5
11	writer-B	0	INC-4412	2
12	writer-A	1	INC-4412	3
13	writer-A	1	INC-2393	4
14	writer-B	0	INC-5457	4

15	writer-A	1	INC-2393	5
16	writer-A	1	INC-4412	5
17	writer-B	0	INC-5457	6
18	writer-B	0	INC-8206	2
19	writer-B	0	INC-4412	4
20	writer-B	0	INC-2393	6

Run 2 - key = incident id: order in which the 20 messages were processed

#	writer	partition	incident	seq
1	writer-B	1	INC-2393	1
2	writer-A	0	INC-5457	1
3	writer-B	1	INC-2393	2
4	writer-A	0	INC-8446	1
5	writer-B	1	INC-2393	3
6	writer-B	1	INC-2393	4
7	writer-A	0	INC-5457	2
8	writer-B	1	INC-2393	5
9	writer-B	1	INC-2393	6
10	writer-A	0	INC-4412	1
11	writer-A	0	INC-8206	1
12	writer-A	0	INC-4412	2
13	writer-A	0	INC-5457	3
14	writer-A	0	INC-5457	4
15	writer-A	0	INC-5457	5
16	writer-A	0	INC-5457	6
17	writer-A	0	INC-4412	3
18	writer-A	0	INC-8206	2
19	writer-A	0	INC-4412	4
20	writer-A	0	INC-4412	5

```
Pragnaya Priyadarshini HW8 — failure_tests.py ordering

================================================================================
PART 3.IV  ORDERING — 20 related messages from VERIFY_SEED=263170, 2 partitions, 2 writers in one group
================================================================================
  --- [UNKEYED] the two writers' partition assignments
      11:05:55.245 [writer-A] Consumer is ready and listening... group=s3170_ord_unkeyed_1031110554 partitions=[0] mode=stub idempotent=0 commit=after max_retries=3 dlq=1
      11:05:55.245 [writer-B] Consumer is ready and listening... group=s3170_ord_unkeyed_1031110554 partitions=[1] mode=stub idempotent=0 commit=after max_retries=3 dlq=1
11:05:58.397 [TEST    ] [UNKEYED] sent 20 messages (NO key, round-robin); partitions used per entity: INC-2393:[0, 1], INC-4412:[0, 1], INC-5457:[0, 1], INC-8206:[0, 1], INC-8446:[1]
  --- [UNKEYED] order in which messages were PROCESSED (time-sorted across both writers)
      #  time      writer    partition  entity    seq
      1  11:05:58.530  writer-B          1  INC-8446    1
      2  11:05:58.695  writer-A          0  INC-5457    1
      3  11:05:58.786  writer-A          0  INC-2393    1
      4  11:05:58.916  writer-B          1  INC-5457    2
      5  11:05:59.068  writer-A          0  INC-4412    1
      6  11:05:59.076  writer-B          1  INC-8206    1
      7  11:05:59.136  writer-B          1  INC-2393    3
      8  11:05:59.385  writer-B          1  INC-5457    3
      9  11:05:59.402  writer-A          0  INC-2393    2
     10  11:05:59.508  writer-B          1  INC-5457    5
     11  11:05:59.703  writer-A          0  INC-4412    2
     12  11:05:59.778  writer-B          1  INC-4412    3
     13  11:05:59.890  writer-B          1  INC-2393    4
     14  11:05:59.989  writer-A          0  INC-5457    4
     15  11:06:00.167  writer-B          1  INC-2393    5
     16  11:06:00.229  writer-B          1  INC-4412    5
     17  11:06:00.351  writer-A          0  INC-5457    6
     18  11:06:00.679  writer-A          0  INC-8206    2
     19  11:06:00.729  writer-A          0  INC-4412    4
     20  11:06:00.847  writer-A          0  INC-2393    6
11:06:00.919 [TEST    ] [UNKEYED] INC-2393: partitions=[0, 1] processed seq order=[1, 3, 2, 4, 5, 6] -> OUT OF ORDER
11:06:00.919 [TEST    ] [UNKEYED] INC-4412: partitions=[0, 1] processed seq order=[1, 2, 3, 5, 4] -> OUT OF ORDER
11:06:00.919 [TEST    ] [UNKEYED] INC-5457: partitions=[0, 1] processed seq order=[1, 2, 3, 5, 4, 6] -> OUT OF ORDER
11:06:00.919 [TEST    ] [UNKEYED] INC-8206: partitions=[0, 1] processed seq order=[1, 2] -> IN ORDER
11:06:00.919 [TEST    ] [UNKEYED] INC-8446: partitions=[1] processed seq order=[1] -> IN ORDER
  --- [KEYED] the two writers' partition assignments
      11:06:02.590 [writer-A] Consumer is ready and listening... group=s3170_ord_keyed_1031110554 partitions=[1] mode=stub idempotent=0 commit=after max_retries=3 dlq=1
      11:06:02.590 [writer-B] Consumer is ready and listening... group=s3170_ord_keyed_1031110554 partitions=[0] mode=stub idempotent=0 commit=after max_retries=3 dlq=1
11:06:05.726 [TEST    ] [KEYED] sent 20 messages (key = entity); partitions used per entity: INC-2393:[1], INC-4412:[0], INC-5457:[0], INC-8206:[0], INC-8446:[0]
  --- [KEYED] order in which messages were PROCESSED (time-sorted across both writers)
      #  time      writer    partition  entity    seq
      1  11:06:05.796  writer-A          1  INC-2393    1
      2  11:06:06.020  writer-B          0  INC-5457    1
      3  11:06:06.129  writer-A          1  INC-2393    2
      4  11:06:06.163  writer-B          0  INC-8446    1
      5  11:06:06.190  writer-A          1  INC-2393    3
      6  11:06:06.303  writer-A          1  INC-2393    4
      7  11:06:06.547  writer-B          0  INC-5457    2
      8  11:06:06.581  writer-A          1  INC-2393    5
      9  11:06:06.700  writer-A          1  INC-2393    6
     10  11:06:06.830  writer-B          0  INC-4412    1
     11  11:06:06.989  writer-B          0  INC-8206    1
     12  11:06:07.293  writer-B          0  INC-4412    2
     13  11:06:07.542  writer-B          0  INC-5457    3
     14  11:06:07.830  writer-B          0  INC-5457    4
     15  11:06:07.959  writer-B          0  INC-5457    5
     16  11:06:08.323  writer-B          0  INC-5457    6
     17  11:06:08.591  writer-B          0  INC-4412    3
     18  11:06:08.919  writer-B          0  INC-8206    2
     19  11:06:08.968  writer-B          0  INC-4412    4
     20  11:06:09.035  writer-B          0  INC-4412    5
11:06:09.255 [TEST    ] [KEYED] INC-2393: partitions=[1] processed seq order=[1, 2, 3, 4, 5, 6] -> IN ORDER
11:06:09.255 [TEST    ] [KEYED] INC-4412: partitions=[0] processed seq order=[1, 2, 3, 4, 5] -> IN ORDER
11:06:09.255 [TEST    ] [KEYED] INC-5457: partitions=[0] processed seq order=[1, 2, 3, 4, 5, 6] -> IN ORDER
11:06:09.255 [TEST    ] [KEYED] INC-8206: partitions=[0] processed seq order=[1, 2] -> IN ORDER
11:06:09.256 [TEST    ] [KEYED] INC-8446: partitions=[0] processed seq order=[1] -> IN ORDER
11:06:10.091 [TEST    ] RESULT ordering: unkeyed -> 3 of 5 entities out of order; keyed -> 0 out of order, 5/5 entities in a single partition -> PASS

================================================================================
SUMMARY
```

[Screenshot 3.4: both ordering runs - processing order and per-incident verdict]

V. Comparison with the HW2 in-process graph

In HW2 the graph was function calls inside one process, each step waiting on the previous one. With Kafka:

Two benefits gained.

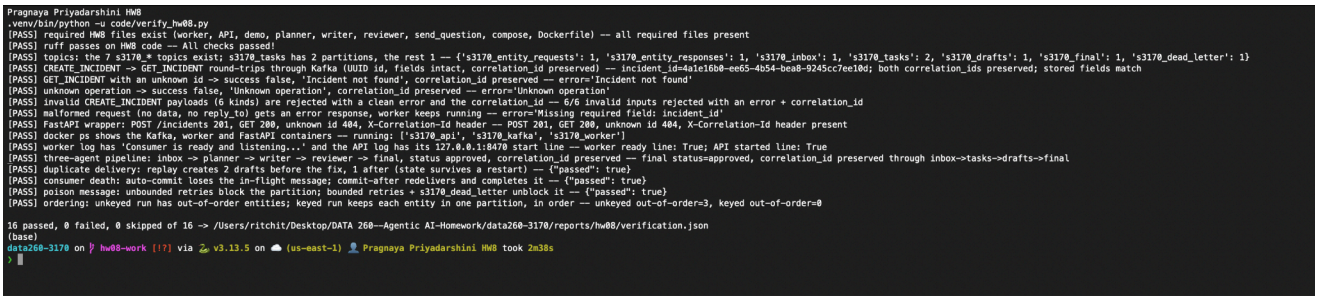
- Independent scaling: the Writer (slow, model-bound) can be run as 2 instances sharing the s3170_tasks partitions while the Planner and Reviewer stay single - impossible when every step is a call in one process.
- Decoupled failure: when the Writer was killed mid-work, the Planner and Reviewer were unaffected and the broker held the message until the Writer came back; in the HW2 graph, one crashed step would have taken the whole run down.

Two new responsibilities I now own.

- Duplicate delivery: Kafka is at-least-once, so the Writer must make processing idempotent (a replayed correlation_id must not repeat the side effect); in-process calls never delivered a step twice.
- Ordering and offsets: order exists only within a partition, so related messages need a key; and the offset must be committed after the work, not before, or a crash silently loses the message. (The same applies to poison messages: bounded retries and a dead-letter topic are mine to build.)

Self-check and repository

make verify-hw08 checks, against the running stack, that CREATE_INCIDENT → GET_INCIDENT round-trips through Kafka with the correlation_id preserved, that errors and invalid input are rejected cleanly, that the API wrapper works, that the three containers and the required log lines are present, that a stub-mode pipeline reaches s3170_final, and that all four failure scenarios behave as reported (16 passed, 0 failed, 0 skipped of 16). Deliverables in reports/hw08/: RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json and this report.



[Screenshot 4.1: terminal - make verify-hw08]

AI use statement

(1) what you used an AI assistant for and what you did yourself

I completed the implementation, testing, debugging, and verification for this assignment myself. I used Claude as a reference when I needed help understanding Kafka consumer behavior, idempotency handling, retry logic, partition offsets, interpreting error messages, and making a few minor fixes. Claude provided suggestions and explanations, but I wrote and integrated the code, made the design decisions, ran the experiments, investigated the failures, and verified the final results myself.

I independently analyzed the duplicate-delivery problem, identified the race condition caused by the shared temporary state file, changed the Writer state to an append-only log, adjusted the retry and bookkeeping order, corrected the test harness's start-offset and partition handling, investigated the failed verification runs, and confirmed the final results using two consecutive full runs and make verify-08.

(2) one AI-produced output that was wrong/unsuitable, or one thing you independently verified

The first version of my Writer, created with some AI assistance, stored its idempotency state in one JSON file and rewrote it through a shared temporary filename. When I ran two Writers in the same consumer group, they competed for that file, and one process failed with an ENOENT error. The retry loop then repeated the entire message, causing the draft to be delivered twice. This exposed the duplicate-delivery problem addressed in Part 3.

I also found problems in the first versions of my failure tests. The "ready" message was printed before the consumer had fixed its starting offset, and the poison and death tests were sending messages to incorrect partitions.

(3) how you detected the problem or verified the result

I detected the main problem by examining the ordering-test results. Although 20 messages were sent, the test processed 21 messages. The processing log showed that the same partition and offset had been handled twice, with a RETRY caused by [Errno 2] immediately beforehand.

I also inspected the failed death and poison test runs and found that no offset had been committed in the consumer-group table. While retaking the screenshots, I received additional FAIL results. I investigated the running processes and found two Writer processes left over from an interrupted run. After stopping those processes, I reran the tests and confirmed that they passed. I kept the earlier failed runs in raw/dev_history/ for comparison and documentation.

(4) what you changed and why it works now.

I changed the Writer's idempotency state from a shared rewritten JSON file to an append-only log containing one correlation_id per line. This allows multiple Writer instances to share the state more safely without racing over one temporary file.

I also changed the retry structure so that retries cover only the processing work and draft delivery. The bookkeeping is performed afterward, which prevents the entire message from being unnecessarily repeated because of a state-file error.

For the test harness, I made it establish the starting offsets before printing "ready" and ensured that related test messages are sent to the same partition. I then ran all four scenarios twice consecutively and confirmed that both full runs passed. The final verification command, make verify-hw08, passed all 16 of 16 checks.

The remaining limitations are that the tests use stub drafts, Kafka runs with a single broker, and delivery is at-least-once. Therefore, a crash occurring after draft delivery but before the delivery is recorded could still result in one duplicate.

References

- Apache Kafka documentation: https://kafka.apache.org/documentation/ (consumer groups, offsets, partitioning by key).
- confluent-kafka-python: https://docs.confluent.io/kafka-clients/python/current/overview.html